SYSTEM DESIGN CONCEPTS WITH EXAMPLES

Read the question, explain the concept in your own words, and give an example before uncovering the answer. Mark the boxes after a first correct explanation, next-day recall, and seven-day recall. Every entry includes the purpose, trade-off, and a concrete example.

1. Foundations and Core Trade-offs

C001 What is scalability? [ ] [ ] [ ]

Answer: Scalability is a system's ability to handle more users, requests, data, or work while keeping performance and reliability within acceptable limits. Vertical scaling adds resources to one machine and is operationally simple but has a hard ceiling. Horizontal scaling adds machines and improves fault tolerance, but requires load balancing, stateless request handling, partitioning, and coordination. Good scalability is not merely accepting more traffic; it keeps cost and operational complexity proportional to useful work.

Example: *A Spring Boot API may first move from 2 to 8 CPU cores, then later run twenty replicas behind a load balancer while PostgreSQL, Redis, and Kafka are scaled independently.*

C002 What is the difference between stateless and stateful services? [ ] [ ] [ ]

Answer: A stateless service keeps no required client session or workflow state only in one process, so any healthy instance can handle the next request. A stateful service owns durable or connection-specific state, making routing, recovery, and scaling more complicated. Stateless does not mean the application has no data; it means durable state is stored in shared databases, caches, object storage, or message systems rather than being trapped in one instance's memory.

Example: *An authentication API that validates a JWT and reads PostgreSQL can be stateless. A WebSocket gateway holding thousands of live connections is stateful at the connection layer even if messages are durably stored elsewhere.*

C003 What is the difference between latency and throughput? [ ] [ ] [ ]

Answer: Latency is the time one operation takes, while throughput is the amount of work completed per unit of time. They influence each other but are not interchangeable. Batching can improve throughput by processing many items together while increasing the waiting time for each item. High concurrency can raise throughput until a resource saturates, after which queueing causes latency to rise sharply. System design should name both targets and use percentiles rather than only averages.

Example: *A media worker may process 1,000 images per minute in batches but make one thumbnail wait 20 seconds. A feed API may target 300 ms p95 latency and 20,000 requests per second.*

C004 How do availability, reliability, and durability differ? [ ] [ ] [ ]

Answer: Availability is whether the system can serve a request now. Reliability is whether it continues producing correct behavior over time. Durability is whether acknowledged data survives failures. A service may be available but unreliable if it returns incorrect results, or available but not durable if it accepts a write that disappears after a crash. Architecture should state which property matters for each operation because improving one can increase cost or reduce another.

Example: *A cached feed can remain available during a database outage but show stale data. A payment ledger may reject new writes during quorum loss to protect reliability and durability rather than return a misleading success.*

C005 What are SLI, SLO, SLA, and error budget? [ ] [ ] [ ]

Answer: A service-level indicator is a measured behavior such as successful-request rate or p95 latency. A service-level objective is the internal target for that indicator. A service-level agreement is an external commitment that may include penalties. The error budget is the allowed unreliability implied by the SLO and helps balance feature delivery against stability work. Targets should describe user-visible outcomes and a time window, not only server health.

Example: *If the feed-read SLO is 99.9 percent successful requests per month, the remaining 0.1 percent is the error budget. Rapid budget burn may pause risky releases even while the formal customer SLA is still being met.*

C006 What does the CAP theorem actually say? [ ] [ ] [ ]

Answer: CAP says that when a network partition prevents nodes from communicating, a distributed system must choose between maintaining a single consistent view and remaining available for every request. It does not say a system permanently chooses only two of three properties under normal conditions. Partition tolerance is unavoidable once data spans nodes, so the meaningful question is what each operation does during a partition: reject or delay, or accept potentially conflicting state.

Example: *During a partition, a payment balance may reject withdrawals to avoid overspending, while an Instagram like counter may accept regional updates and reconcile them later because temporary disagreement is tolerable.*

C007 What does PACELC add to CAP? [ ] [ ] [ ]

Answer: PACELC extends CAP by describing the normal case. If a partition occurs, a system chooses availability or consistency; else, when there is no partition, it still trades latency against consistency. Stronger replication often waits for more nodes and adds network delay, while lower-latency reads may use local replicas and return older data. PACELC is useful because partitions are rare, but the latency-consistency decision affects every request.

Example: *A globally consistent username reservation may wait for a cross-region quorum even on a healthy day. A feed read may use the nearest replica for lower latency and accept that a new post appears a few seconds later.*

C008 What is a consistency model? [ ] [ ] [ ]

Answer: A consistency model defines what values clients are allowed to observe when operations occur concurrently or replicas update at different times. Examples include eventual consistency, causal consistency, read-your-writes, monotonic reads, and linearizability. The model is a behavioral contract, not simply a database setting. Different fields in one product can use different models based on the cost of stale or conflicting data.

Example: *A profile editor may need read-your-writes so the user sees a saved biography immediately, while public follower counts can be eventually consistent. Account ownership may require linearizable changes so two users never receive the same username.*

C009 What is eventual consistency? [ ] [ ] [ ]

Answer: Eventual consistency means replicas may temporarily return different values after a write, but if updates stop and communication continues, they converge to the same state. It improves availability, geographic latency, and write throughput because every operation does not wait for all replicas. The design still needs conflict rules, versioning, retry behavior, and a bound or expectation for convergence. Eventual consistency is not random or permanently incorrect data; it is an explicit period during which different observations are allowed.

Example: *After a user likes a post, their screen can update immediately while the public like counter is asynchronously recomputed. Other users may briefly see the old count, but events and reconciliation eventually produce the same value everywhere.*

C010 What are strong consistency and linearizability? [ ] [ ] [ ]

Answer: Strong consistency usually means a read reflects the latest completed write according to the system's contract. Linearizability is a precise form: every operation appears to occur atomically at one point between its start and completion, respecting real-time order. It simplifies reasoning but requires coordination, which adds latency and may reduce availability during partitions. Use it for invariants where stale or conflicting values would create incorrect ownership or financial state.

Example: *If one request successfully reserves username `alex`, a later request must observe it as unavailable. Sending the second request to a lagging replica would violate a linearizable uniqueness decision.*

2. Transactions, Concurrency, and Lock Types

C011 What do the ACID properties mean? [ ] [ ] [ ]

Answer: Atomicity means a transaction's changes commit together or not at all. Consistency means the transaction preserves declared database invariants. Isolation controls how concurrent transactions observe one another. Durability means committed changes survive failures according to the storage guarantee. ACID protects a transaction boundary, but it does not automatically make an entire distributed workflow correct when a database, broker, and external API participate separately.

Example: *Creating an order and its line items can commit in one PostgreSQL transaction. Sending a confirmation email is outside that transaction, so an outbox event is needed to reliably connect the database commit to asynchronous delivery.*

C012 What are transaction isolation levels? [ ] [ ] [ ]

Answer: Isolation levels define which effects of concurrent transactions are visible. Read uncommitted allows the weakest visibility. Read committed prevents dirty reads but may return different values during one transaction. Repeatable read keeps previously read rows stable, though exact phantom behavior depends on the database. Serializable makes committed transactions behave as if executed one at a time, but may reduce concurrency or abort conflicts. Choose the weakest level that still preserves the business invariant.

Example: *A report may use read committed, while reserving the last inventory unit may require a conditional atomic update or serializable transaction so two checkouts cannot both succeed.*

C013 What are dirty reads, non-repeatable reads, and phantom reads? [ ] [ ] [ ]

Answer: A dirty read observes data another transaction has not committed and may later roll back. A non-repeatable read occurs when the same row returns a different committed value within one transaction. A phantom occurs when repeating a range query returns newly inserted or removed matching rows. These anomalies describe observable behavior; whether they are harmful depends on the workflow. Stronger isolation prevents more anomalies but increases locking, retries, or version-management cost.

Example: *A checkout that counts available seats twice may see a phantom reservation inserted between queries. A statement based on the first count can become incorrect unless the invariant is enforced atomically.*

C014 What is MVCC? [ ] [ ] [ ]

Answer: Multi-Version Concurrency Control keeps multiple row versions so readers can view a consistent snapshot without blocking ordinary writers. Each transaction sees versions allowed by its snapshot and isolation level, while obsolete versions are later cleaned up. MVCC improves concurrency but does not remove all conflicts: concurrent writes to the same row, long-running snapshots, and uniqueness constraints still need handling. Old versions also consume space until cleanup can reclaim them.

Example: *PostgreSQL can let a long report read a stable snapshot while users update profiles. If the report runs for hours, old row versions may accumulate and delay vacuum cleanup even though writers are not directly blocked.*

C015 What is optimistic locking? [ ] [ ] [ ]

Answer: Optimistic locking assumes conflicts are uncommon and does not hold a lock while a user or service prepares an update. Each record carries a version. The update succeeds only if the stored version still matches the version originally read; otherwise the caller receives a conflict and retries, merges, or asks the user. It provides high concurrency for short writes but performs poorly when many actors repeatedly update the same hot record.

Example: *A profile is read at version 12. The update executes `WHERE id = ? AND version = 12` and increments to 13. A second edit based on version 12 updates zero rows and must reload.*

C016 What is pessimistic locking? [ ] [ ] [ ]

Answer: Pessimistic locking assumes a conflict is likely or too costly, so a transaction locks data before changing it and makes competitors wait or fail. Row locks limit the affected scope, while broader locks reduce concurrency. Locks must be held for the shortest possible transaction and never across slow user input or external network calls. Pessimistic locking can protect hot invariants but introduces blocking, timeout, and deadlock risks.

Example: *A worker may select one pending job `FOR UPDATE SKIP LOCKED`, mark it claimed, and commit quickly. It should not hold that database lock while performing a five-minute video conversion.*

C017 What are shared, exclusive, and read-write locks? [ ] [ ] [ ]

Answer: A shared or read lock allows multiple readers but blocks an incompatible writer. An exclusive or write lock allows one writer and prevents other readers or writers according to the implementation. A read-write lock is an in-process abstraction that permits concurrent readers or one writer. These locks are useful only when every participant respects the same lock and the protected state cannot change through another path. Long read sections can starve writers unless fairness is configured.

Example: *An in-memory configuration map may use a read-write lock so thousands of requests read concurrently while a rare refresh obtains exclusive access to replace the map safely.*

C018 What are row, table, and intention locks? [ ] [ ] [ ]

Answer: Row locks protect specific records and allow unrelated rows to change concurrently. Table locks protect a whole table and are simpler but greatly reduce concurrency. Intention locks announce that a transaction holds or plans lower-level locks, allowing the database to check compatibility between table-level and row-level operations efficiently. Developers normally express SQL operations while the database chooses lock details, but query shape and indexes influence how many rows are scanned and locked.

Example: *Updating one indexed account row may lock only that row. An unindexed update that scans a large range can lock many records and block unrelated requests even though the application intended a narrow change.*

C019 What is a distributed lock, and why are fencing tokens important? [ ] [ ] [ ]

Answer: A distributed lock coordinates processes on different machines using a shared service and usually a time-limited lease. A lease can expire while an old holder is paused, allowing a new holder to start while the old one later resumes. A fencing token solves this by assigning each lease a larger sequence number; the protected resource rejects operations from older tokens. Without resource-side validation, the lock service cannot prevent a stale worker from writing.

Example: *Worker A receives token 41, pauses, and loses its lease. Worker B receives 42 and writes. When A resumes, storage rejects token 41 instead of allowing stale output to overwrite B.*

C020 What is a deadlock? [ ] [ ] [ ]

Answer: A deadlock occurs when transactions form a cycle of waiting: each holds a resource another needs, so none can continue. Databases detect the cycle and abort one participant, but the application must handle that error safely. Prevention includes acquiring resources in a consistent order, shortening transactions, indexing lookups, and moving external calls outside the transaction. Retrying with jitter handles a victim abort but does not remove a frequent structural problem.

Example: *Transaction A locks user 1 then requests user 2, while transaction B locks user 2 then requests user 1. Always locking the smaller user ID first removes that cycle.*

3. Databases, Indexes, Replication, and Sharding

C021 What are normalization and denormalization? [ ] [ ] [ ]

Answer: Normalization separates data into related structures to reduce duplication and update anomalies. Denormalization intentionally duplicates or prejoins data to make important reads faster or simpler. Normalized models suit transactional correctness and changing relationships, while denormalized views suit read-heavy access patterns. Duplicated data needs an owner, update path, staleness policy, and repair process; otherwise faster reads create silent inconsistency.

Example: *User identity can remain normalized in a `users` table, while a feed entry stores a snapshot of display name and thumbnail for fast rendering. A profile-change event later refreshes those feed snapshots.*

C022 What is a database index? [ ] [ ] [ ]

Answer: An index is an additional data structure that lets the database locate rows without scanning the entire table. It speeds selected reads and ordering but consumes storage and adds work to inserts, updates, deletes, replication, and backups. An index is useful only when its key order and selectivity match real query predicates. Too many indexes can make a write-heavy system slower even if each index helps one rare query.

Example: *An index on `(owner_id, created_at)` lets a profile page find one user's newest posts efficiently. An index only on `created_at` may still scan posts from many owners before filtering.*

C023 What are composite and covering indexes? [ ] [ ] [ ]

Answer: A composite index orders several columns together, so its leftmost key sequence determines which predicates and sorts it can serve efficiently. A covering index contains every column needed by a query, allowing the database to answer from the index without fetching table rows. Covering improves reads but makes the index larger and writes more expensive. Column order should follow equality filters, ranges, ordering, and actual execution plans rather than a memorized rule alone.

Example: *`(user_id, created_at DESC) INCLUDE (caption)` can fetch a user's recent captions in order without touching the post table, but it does not efficiently answer a query filtering only by `created_at`.*

C024 What is a query execution plan? [ ] [ ] [ ]

Answer: An execution plan is the database's chosen sequence of scans, index lookups, joins, sorts, and aggregations for a query. The optimizer estimates row counts and costs from statistics; wrong estimates or changed data distribution can produce a poor plan. Investigation should use the real query shape and parameters and compare estimated with actual rows. Adding an index without reading the plan may not help and can increase write cost.

Example: *A query expected to return ten rows may actually return ten million, causing a nested-loop join to explode. Updated statistics or a different join and index can make the plan appropriate again.*

C025 What is database replication? [ ] [ ] [ ]

Answer: Replication keeps copies of data on multiple nodes for availability, disaster recovery, read scaling, or geographic access. A leader-follower design sends writes to one leader and replays them on followers. Multi-leader and leaderless systems allow broader writes but need conflict and quorum rules. Replication is not a backup: an accidental delete or corrupt write can quickly propagate to every replica. Lag, failover, and promotion behavior must be measured and tested.

Example: *PostgreSQL may stream write-ahead logs to a standby for failover and to read replicas for feed queries, while separate point-in-time backups protect against logical deletion.*

C026 How do synchronous and asynchronous replication differ? [ ] [ ] [ ]

Answer: Synchronous replication waits for one or more replicas before acknowledging a write, improving durability and failover freshness at the cost of latency and availability when replicas are unreachable. Asynchronous replication acknowledges earlier and ships changes later, improving latency and write availability but allowing lag and possible loss of acknowledged writes if the leader fails. Some systems use a mix, synchronously protecting a nearby replica and asynchronously copying to distant regions.

Example: *A payment ledger may wait for a second availability-zone replica, while feed analytics replicate asynchronously across regions because losing a few seconds can be repaired from events.*

C027 What is replication lag and how does it affect reads? [ ] [ ] [ ]

Answer: Replication lag is the delay between a committed source write and its application on a replica. Reads from a lagging replica can miss recent data, violate read-your-writes, or appear to move backward when requests switch replicas. The system can route freshness-sensitive reads to the primary, wait for a known version, or use session affinity. Lag should be measured in both time and remaining work because a quiet period can make one measure misleading.

Example: *After updating a profile, the API routes that user's reads to the primary for several seconds, while unrelated public profile views continue using replicas.*

C028 What are partitioning and sharding? [ ] [ ] [ ]

Answer: Partitioning divides data into smaller pieces using a key or time range. Sharding usually means those pieces live on different database nodes, allowing storage and traffic to scale beyond one machine. Partitioning can also occur inside one database for management and pruning. Distribution introduces routing, rebalancing, cross-shard joins, transactions, backup coordination, and hotspot risks. It should solve a measured limit rather than be added only because the system might grow someday.

Example: *Posts can be partitioned by month inside PostgreSQL for retention, while user data can be sharded by user ID across several database clusters when one primary can no longer handle writes.*

C029 What makes a good shard key? [ ] [ ] [ ]

Answer: A shard key should have high cardinality, distribute storage and request load evenly, remain stable, and colocate data commonly accessed or transacted together. A key with many values can still be poor if one value is extremely hot. Changing the key later is expensive, so the design needs a routing directory and resharding plan. Cross-shard operations should be identified before selection, not discovered after migration.

Example: *User ID often distributes normal social accounts and colocates each user's private data. A celebrity can still become a hot key, so public posts and counters may need subpartitioning or separate caching.*

C030 What is consistent hashing? [ ] [ ] [ ]

Answer: Consistent hashing maps keys and nodes onto a logical ring so adding or removing a node moves only a fraction of keys instead of remapping almost everything. Virtual nodes give each physical node many positions and improve balance. It is useful for distributed caches and partition ownership, but it does not automatically solve heterogeneous capacity, hot keys, replication, or safe movement. Operational systems still need membership, health, transfer, and failure rules.

Example: *When a Redis-like cache cluster grows from ten to eleven nodes, consistent hashing may move roughly one eleventh of keys rather than invalidating nearly the entire cache.*

4. Caching and Content Delivery

C031 What is cache-aside? [ ] [ ] [ ]

Answer: In cache-aside, the application checks the cache, loads missing data from the source, stores it, and returns it. Writes normally update the source and invalidate the cache. The pattern is simple and keeps the database authoritative, but the first miss is slow, concurrent misses can create a stampede, and stale data can remain after invalidation failure. Keys, TTL, negative caching, and fallback limits must be designed explicitly.

Example: *A profile API checks Redis for `profile:42`; on a miss it reads PostgreSQL and caches the result. A profile update commits to PostgreSQL and then deletes `profile:42`.*

C032 How do read-through and write-through caches work? [ ] [ ] [ ]

Answer: A read-through cache loads missing values from the source on behalf of the application, centralizing miss behavior. A write-through cache synchronously writes both cache and backing store before confirming success, making cached reads immediately current when both succeed. These patterns simplify callers but make the cache layer more responsible for data access and failure handling. Write-through increases write latency and still needs a clear response when one destination succeeds and the other fails.

Example: *A product catalog library may expose one read-through interface. Price updates use write-through only if the cache provider and database update contract is understood and failures are repairable.*

C033 What is write-behind caching? [ ] [ ] [ ]

Answer: Write-behind updates the cache first and persists changes to the backing store asynchronously. It can reduce perceived write latency and batch many updates, but unflushed data may be lost if the cache fails, and ordering or duplicate writes must be handled. It suits data that tolerates delayed durability or can be reconstructed, not critical state whose successful response promises permanent storage.

Example: *View counts can accumulate in Redis and flush to a database every few seconds with idempotent batches. A bank transfer should not be acknowledged solely from a volatile write-behind cache.*

C034 Why is cache invalidation difficult? [ ] [ ] [ ]

Answer: Cached values can exist in browser, CDN, application memory, distributed cache, and derived read models. A source update must either invalidate or version every relevant copy, and races can repopulate stale data after deletion. TTL limits the duration of mistakes but does not guarantee freshness. Immutable versioned URLs, event-driven invalidation, source versions, and reconciliation reduce ambiguity. The policy should state acceptable staleness rather than claiming the cache is always current.

Example: *Replacing an avatar should create a new versioned media URL. Reusing the same URL may leave old bytes in browsers and CDN edges even after Redis is cleared.*

C035 What is cache TTL? [ ] [ ] [ ]

Answer: Time to live is how long a cached entry remains valid before expiring. A longer TTL improves hit rate and protects the origin but increases the possible stale-data window. A shorter TTL improves freshness but creates more misses and dependency load. TTL should follow update frequency, recomputation cost, security sensitivity, and product tolerance, with randomized jitter to prevent many entries expiring together.

Example: *Public immutable media can use a one-year TTL with versioned URLs, profile summaries may use minutes, and permission decisions may be uncached or invalidated immediately with a short safety TTL.*

C036 What are LRU, LFU, and FIFO eviction policies? [ ] [ ] [ ]

Answer: Least Recently Used evicts entries not accessed for the longest time and works well when recent use predicts future use. Least Frequently Used favors historically popular entries but can keep formerly hot data too long unless frequency decays. First In First Out removes the oldest inserted entry and is simple but ignores usefulness. Real caches also consider entry size, TTL, and memory overhead. Eviction policy cannot compensate for a key space larger than useful cache capacity.

Example: *A feed-object cache may prefer LRU for changing user activity, while a stable dictionary with consistently popular keys may benefit from LFU with aging.*

C037 What is a cache stampede or thundering herd? [ ] [ ] [ ]

Answer: A cache stampede occurs when many requests miss the same popular key and simultaneously recompute or fetch it, overwhelming the origin. Prevention includes request coalescing, a short distributed loader lock, proactive refresh, stale-while-revalidate, randomized TTL, and bounded fallback concurrency. The waiting path needs a timeout because one failed loader must not block every request indefinitely.

Example: *When a viral-post entry expires, one request refreshes it from PostgreSQL while other requests receive the slightly stale value or wait briefly, rather than issuing thousands of identical queries.*

C038 What are cache penetration and negative caching? [ ] [ ] [ ]

Answer: Cache penetration occurs when requests repeatedly ask for nonexistent keys, so every lookup reaches the database. Negative caching stores a short-lived not-found result to protect the origin. The cache must distinguish genuine absence from dependency errors, and creation should invalidate the negative entry. Attackers can still generate unlimited unique keys, so input validation and rate limiting remain necessary.

Example: *Repeated requests for nonexistent username `xyz123` can cache `NOT_FOUND` for thirty seconds. If that username is created, the application deletes the negative entry immediately.*

C039 What is a hot key? [ ] [ ] [ ]

Answer: A hot key receives a disproportionate share of traffic, concentrating load on one cache node, shard, or lock even when the overall cluster is underutilized. More normal partitions do not spread one key. Solutions include CDN or local caching, replication of read-only hot values, request coalescing, subpartitioning, batching, and rate limits. Hot-key detection requires per-key or heavy-hitter metrics rather than cluster averages alone.

Example: *A celebrity post can overload the one Redis shard that owns its metadata. Edge caching and short local caches can absorb reads before the distributed cache.*

C040 What does a CDN do? [ ] [ ] [ ]

Answer: A Content Delivery Network stores or proxies content at geographically distributed edge locations near users. It reduces latency, origin bandwidth, and load, and can provide TLS termination, request filtering, and DDoS absorption. CDNs work best with immutable or safely cacheable responses. Personalized content requires cache-key and authorization care, and purges are not as simple as updating one database row.

Example: *Original photos and generated thumbnails use versioned object URLs cached at edges for months. The personalized home-feed JSON remains private and uses short or no shared caching.*

5. Messaging, Queues, and Asynchronous Work

C041 What is the difference between a queue and a stream? [ ] [ ] [ ]

Answer: A queue distributes work so a message is normally handled by one worker, while a stream is an ordered durable log that multiple independent consumer groups can read at their own positions. Queue systems emphasize task ownership and acknowledgement; stream systems emphasize retention, replay, partition order, and event history. Products may offer both behaviors, so the important question is the consumption and retention contract rather than the brand name.

Example: *Image-resize jobs can use a work queue where one worker owns each job. Post-created events can use a stream so feed, search, analytics, and notification consumers each process the same event independently.*

C042 What is publish-subscribe messaging? [ ] [ ] [ ]

Answer: Publish-subscribe lets producers publish an event without naming every consumer. Subscribers receive events for topics or patterns and evolve independently. It reduces direct coupling but introduces asynchronous delivery, schema compatibility, duplicate handling, ordering limits, and harder end-to-end tracing. A publisher should describe a completed business fact rather than issue vague commands to unknown consumers, and consumers should own their retry and failure behavior.

Example: *After a post commits, the service publishes `PostCreated`. Search indexes it, feed workers create candidates, analytics counts it, and notifications decide whether followers should be alerted.*

C043 What do at-most-once, at-least-once, and exactly-once mean? [ ] [ ] [ ]

Answer: At-most-once may lose a message but does not intentionally redeliver it. At-least-once retries until acknowledgement, so duplicates are possible. Exactly-once claims require a precise boundary: a broker may process a record once internally while an external database or email provider still sees duplicates after uncertain failures. Most robust systems use at-least-once delivery with idempotent business effects rather than assuming universal exactly-once behavior.

Example: *A notification consumer can receive one event twice but store a unique notification ID, so the user sees one notification. The duplicate delivery exists, but the user-visible effect is once.*

C044 What is idempotency? [ ] [ ] [ ]

Answer: An idempotent operation produces the same intended result when repeated with the same logical input. It is essential because clients retry timeouts and brokers redeliver messages. Idempotency usually requires a stable request or event ID plus durable uniqueness or state-transition checks. Simply checking memory on one instance is insufficient. The contract must define key scope, retention, and what happens if the same key arrives with a different payload.

Example: *A payment API stores idempotency key `checkout-731` with the charge result. A retry returns the original charge instead of creating another one, even if a different server handles it.*

C045 How is message ordering provided? [ ] [ ] [ ]

Answer: Messaging systems usually guarantee order only within a partition, queue, or key, not across the entire system. A partition key groups related events under one ordered owner, while more partitions increase parallelism. Replays, producer retries, and multiple sources can still require sequence numbers or versions at the consumer. Global ordering is expensive and often unnecessary; define the smallest entity boundary that needs order.

Example: *Profile events are partitioned by user ID so one user's updates remain ordered. Different users update in parallel, and the consumer rejects an older profile version if it arrives late.*

C046 What is a consumer group? [ ] [ ] [ ]

Answer: A consumer group is a set of workers that share responsibility for a subscription. Each partition or message is assigned to one group member at a time, allowing parallel processing without every worker receiving every event. Different groups each receive the stream independently. Rebalancing moves ownership when workers join or leave and can pause work or redeliver records, so handlers must be idempotent and track lag by partition.

Example: *Ten feed workers in one group divide fifty partitions. A separate search group reads all fifty partitions independently to update the search index.*

C047 What is a dead-letter queue? [ ] [ ] [ ]

Answer: A dead-letter queue stores messages that exceeded retry limits or failed with a permanent error, allowing the main queue to continue. It is not a place to forget failures. Each record needs original payload, headers, event ID, schema version, error, attempt history, and an owner. Recovery should fix the cause and replay through an idempotent controlled path. Ordering-sensitive workflows may quarantine one key instead of skipping it globally.

Example: *A malformed `PostCreated` event fails three times, moves to a DLQ, alerts the owning team, and is replayed after the consumer learns the old schema.*

C048 What is the transactional outbox pattern? [ ] [ ] [ ]

Answer: The outbox pattern solves the dual-write problem between a database transaction and a message broker. The service writes its business data and an outbox row in the same local transaction. A relay later publishes committed rows, retrying safely. Consumers remain idempotent because publication can repeat. The pattern adds relay monitoring, cleanup, and delivery delay, but prevents a committed state change from silently missing its event.

Example: *Creating a post inserts both the post and `PostCreated` outbox record atomically. If Kafka is down, the post remains valid and the relay publishes after Kafka recovers.*

C049 What is the saga pattern? [ ] [ ] [ ]

Answer: A saga coordinates a multi-service business workflow as a sequence of local transactions. If a later step fails, compensating actions attempt to undo or offset earlier steps. Choreography uses events, while orchestration uses a coordinator that records state and commands participants. Sagas provide eventual completion, not the isolation of one database transaction, so users may observe pending states and compensations can fail or be only approximate.

Example: *Booking a trip reserves a flight, then a hotel, then payment. If payment fails, the saga sends idempotent commands to release the hotel and flight and records any manual-repair requirement.*

C050 What are backpressure and bounded queues? [ ] [ ] [ ]

Answer: Backpressure prevents producers from sending unlimited work into slower consumers. A bounded queue rejects, delays, or redirects excess work instead of consuming all memory and crashing. The response can propagate upstream through rate limits, reduced concurrency, retry guidance, or paused consumption. A durable broker absorbs bursts only up to retention and storage limits; it cannot solve a permanent arrival rate greater than processing capacity.

Example: *When image workers are saturated, uploads remain accepted only while queue age stays below the limit. After that, the API returns a retryable response or degrades optional processing rather than building an infinite backlog.*

6. Reliability and Resilience Patterns

C051 Why are timeouts necessary? [ ] [ ] [ ]

Answer: A timeout bounds how long a caller waits and how long resources such as threads, connections, and memory remain occupied by uncertain work. Without timeouts, one slow dependency can exhaust the caller. Timeout values should follow the end-to-end latency budget and observed percentiles, with inner calls shorter than outer deadlines. A timeout does not prove the remote operation failed; writes may have committed, so retries need idempotency.

Example: *A feed request with a 500 ms budget gives recommendations 80 ms. If that call times out, the feed uses chronological fallback rather than holding the request for seconds.*

C052 How should retries use exponential backoff and jitter? [ ] [ ] [ ]

Answer: Exponential backoff increases the delay between attempts, reducing pressure on a failing dependency. Jitter adds randomness so many clients do not retry at the same instant. Retries need a maximum attempt count, total deadline, and a decision about which errors are transient. Non-idempotent writes should not retry blindly. Every layer should not independently retry, because attempt counts multiply across the call graph.

Example: *A worker retries a temporary provider error after roughly 1, 2, and 4 seconds with randomized variation, then moves the message to delayed retry instead of looping continuously.*

C053 What is a circuit breaker? [ ] [ ] [ ]

Answer: A circuit breaker stops calls to a dependency when failures or latency cross a threshold. In the open state, calls fail fast or use fallback; after a delay, a limited half-open test checks recovery before normal traffic resumes. It protects caller resources and gives the dependency room to recover. Thresholds need enough volume and hysteresis to avoid flapping, and fallback usage must be monitored as degraded service.

Example: *If the recommendation API times out repeatedly, the feed service opens the circuit and serves chronological content for thirty seconds before allowing a few test calls.*

C054 What is the bulkhead pattern? [ ] [ ] [ ]

Answer: A bulkhead isolates resources so one failing dependency or workload cannot consume everything. Separate thread pools, connection pools, queues, or instance groups limit the blast radius. Isolation reduces shared efficiency and requires capacity decisions, but it preserves critical operations during local overload. Bulkheads work best with timeouts and load shedding; a separate unbounded pool can still fail independently.

Example: *Feed hydration and notification sending use different worker pools. A slow push provider fills the notification pool but cannot consume the threads needed to serve feed reads.*

C055 What are common rate-limiting algorithms? [ ] [ ] [ ]

Answer: Fixed windows count requests in simple time buckets but allow bursts at boundaries. Sliding windows estimate or record activity across the current interval more smoothly. Token buckets refill permits over time and allow controlled bursts. Leaky buckets drain at a steady rate and smooth output. The design must choose identity, scope, distributed accuracy, failure behavior, and a clear retry response, not only an algorithm.

Example: *An upload API may give each user a token bucket of ten uploads with one token restored per minute, allowing a short burst while controlling sustained load.*

C056 What is load shedding? [ ] [ ] [ ]

Answer: Load shedding deliberately rejects or degrades lower-priority work when capacity is exhausted, preventing total collapse. It should occur before queues, threads, or databases are irrecoverably saturated. Requests can be prioritized by endpoint, tenant, user action, freshness, or cost. Rejection should be fast and observable, and callers should not immediately retry without limits. Load shedding protects the core service but does not replace adding capacity or removing the bottleneck.

Example: *During a traffic spike, the system continues login and feed reads but disables expensive recommendations and returns 429 for bulk analytics exports.*

C057 What is graceful degradation? [ ] [ ] [ ]

Answer: Graceful degradation keeps the most important user journey working when optional components fail. It requires predefined fallbacks and a clear distinction between correctness-critical and enhancement features. Degraded responses should remain safe, authorized, and visible to operators. A fallback that silently returns wrong financial or permission data is not graceful. The system should measure how often degradation occurs because a permanent fallback can hide a failed dependency.

Example: *If ranking is unavailable, the home feed returns recent eligible posts. It never skips privacy filters, because authorization is mandatory while ranking quality is optional.*

C058 What is the difference between liveness and readiness checks? [ ] [ ]

Answer: Liveness asks whether the process is stuck and should be restarted. Readiness asks whether it should receive new traffic. A service can be alive but unready because a critical dependency or warm-up is incomplete. Overly deep liveness checks can restart every instance during a shared database outage, making recovery worse. Readiness should use stable signals and hysteresis so brief failures do not create routing flaps.

Example: *A Spring Boot process remains live while Redis is unavailable, but an endpoint that requires Redis may fail readiness or use a fallback. A deadlocked event loop fails liveness and is restarted.*

C059 What is chaos engineering? [ ] [ ] [ ]

Answer: Chaos engineering is the controlled testing of system behavior under failures such as latency, packet loss, process termination, dependency unavailability, or zone loss. It starts with a hypothesis, measurable steady state, limited blast radius, monitoring, and an abort condition. The goal is to discover whether resilience mechanisms and runbooks actually work, not to create random outages. Tests begin in safe environments and progress only with evidence.

Example: *A team slows the recommendation service for five minutes and verifies the feed opens its circuit, uses fallback, keeps p95 latency within target, alerts correctly, and recovers gradually.*

C060 What are RPO and RTO? [ ] [ ] [ ]

Answer: Recovery Point Objective is the maximum acceptable data-loss window measured backward from failure. Recovery Time Objective is the maximum acceptable time to restore service. They drive replication, backup frequency, storage tier, automation, and cost. A backup schedule does not prove either target; restore drills must measure actual data age and complete service recovery. Different data can have different objectives.

Example: *A payment ledger may require near-zero RPO and a fifteen-minute RTO, while derived recommendations can have a one-day RPO because they can be rebuilt from durable events.*

7. APIs, Networking, and Traffic Management

C061 What is REST? [ ] [ ] [ ]

Answer: REST is an architectural style that models resources accessed through a uniform HTTP interface, commonly using methods such as GET, POST, PUT, PATCH, and DELETE. Stateless requests, standard status codes, cache controls, and resource identifiers make integrations understandable. REST does not require JSON and does not automatically produce good APIs; contracts still need bounded payloads, idempotency, authorization, pagination, and compatibility.

Example: *`GET /posts/{id}` retrieves a post, while `DELETE /posts/{id}` removes it only after object-level authorization. Repeating the delete can safely return the already-deleted state.*

C062 What is gRPC and when is it useful? [ ] [ ] [ ]

Answer: gRPC uses Protocol Buffers and HTTP/2 to provide typed contracts, generated clients, compact binary messages, and unary or streaming calls. It is useful for high-volume internal communication and polyglot services that benefit from strict schemas. It adds tooling, browser, proxy, debugging, and versioning considerations. It should be chosen from measured needs; it does not fix slow database queries or excessively chatty service boundaries.

Example: *A media service streams processing progress to an internal orchestrator through gRPC, while the public mobile API continues using JSON over HTTP for broad compatibility.*

C063 What changes with HTTP/2? [ ] [ ] [ ]

Answer: HTTP/2 multiplexes many request streams over one connection, compresses headers, and supports binary framing. It reduces connection overhead and application-layer head-of-line blocking compared with HTTP/1.1 request sequencing. All streams still share TCP congestion and loss recovery, so packet loss can affect the connection. Server and proxy limits, flow control, and long-lived connection behavior remain important.

Example: *A mobile client can fetch several feed resources concurrently over one connection, but a lossy network may still delay multiple streams until TCP retransmits missing packets.*

C064 How do polling, Server-Sent Events, and WebSockets differ? [ ] [ ] [ ]

Answer: Polling repeatedly asks for updates and is simple but wastes requests or increases delay. Server-Sent Events keep one HTTP connection for server-to-client text updates and include reconnection support. WebSockets provide full-duplex messages for interactive real-time systems but require connection routing, heartbeats, backpressure, and reconnect handling. The simplest mechanism that meets direction, frequency, and latency needs is usually best.

Example: *A job-status page may poll every few seconds, a notification panel can use SSE, and chat uses WebSockets because clients and servers both send frequent messages.*

C065 What are DNS and service discovery? [ ] [ ] [ ]

Answer: DNS maps names to addresses and supports public and internal discovery with caching controlled by TTL. Service discovery tracks dynamic service instances, health, ports, and metadata, often through a registry or platform control plane. Clients may resolve through a load balancer, sidecar, or client library. Discovery information is briefly stale by nature, so connection failure and endpoint removal must be handled safely.

Example: *`feed.internal` resolves to a regional load balancer, which routes only to ready feed instances. A terminating instance fails readiness before exit so discovery stops sending new connections.*

C066 What does a load balancer do? [ ] [ ] [ ]

Answer: A load balancer distributes connections or requests across healthy targets, performs health checks, and may terminate TLS. Layer 4 balancing uses transport information such as IP and port, while Layer 7 understands HTTP paths, headers, and methods. Algorithms include round robin, least connections, hashing, and latency-aware routing. A balanced request count can still produce uneven work when payload cost or long-lived connections vary.

Example: *Upload traffic routes to a dedicated pool based on path, while ordinary API traffic uses least-connections routing because upload requests remain active much longer.*

C067 What is an API gateway? [ ] [ ] [ ]

Answer: An API gateway is an edge entry point that centralizes routing and selected cross-cutting concerns such as authentication, TLS, request limits, protocol translation, and observability. It should not become the owner of every business rule or a single scaling bottleneck. Gateway policies need versioning, staged rollout, and high availability because one mistake can affect all services.

Example: *The gateway validates access tokens, limits upload size, and routes `/api/feed` to the feed service. Feed ranking and privacy decisions remain in owned backend services.*

C068 Why is cursor pagination preferred at scale? [ ] [ ] [ ]

Answer: Offset pagination becomes slower on deep pages because the database must find and discard earlier rows, and concurrent inserts can shift results. Cursor or keyset pagination uses the last seen ordered values to fetch the next page with a matching index. It is efficient and stable for forward navigation but does not naturally support jumping to page 5,000. The ordering needs a unique tie-breaker.

Example: *A feed cursor contains `(rank_score, post_id)`. The next query requests rows below that pair, preventing equal scores from duplicating or skipping posts.*

C069 What is an API idempotency key? [ ] [ ] [ ]

Answer: An idempotency key identifies one logical client operation across network retries. The server durably stores the key, request fingerprint, status, and result under a uniqueness constraint. A retry with the same payload returns the original result; reuse with a different payload is rejected. Keys require scope and expiration, and the check plus business write must be atomic.

Example: *A mobile client sends `Idempotency-Key: upload-post-884`. If the response is lost after commit, retrying returns the same post ID instead of creating a duplicate post.*

C070 What is backward compatibility? [ ] [ ] [ ]

Answer: Backward compatibility means a new provider version continues working with older clients and consumers. Additive fields, stable meanings, tolerant readers, defaults, and expand-and-contract migrations support mixed versions. Removing a field, changing its type, or reusing it with new semantics is breaking even if deployment succeeds. Compatibility windows must reflect slow mobile upgrades and asynchronous event replay.

Example: *A response adds optional `mediaStatus` while keeping the old `mediaUrl`. After old mobile usage reaches zero and rollback risk passes, the old field can be retired deliberately.*

8. Distributed Coordination and Ordering

C071 What is consensus? [ ] [ ] [ ]

Answer: Consensus lets distributed nodes agree on an ordered sequence of decisions despite failures. Protocols such as Raft and Paxos use quorums, terms or ballots, and logs so one committed value is not replaced by a conflicting value. Consensus is used for metadata, leader election, configuration, and strongly consistent storage, but coordination adds latency and cannot remain fully available without a quorum.

Example: *A database cluster uses consensus to agree which node is leader and which write-log entries are committed before acknowledging protected writes.*

C072 What is leader election? [ ] [ ] [ ]

Answer: Leader election selects one current coordinator for work that must have a single owner. A robust design uses terms, leases, or quorum decisions so nodes can distinguish newer leadership from stale leadership. Election alone is not enough: a paused old leader may resume, so protected writes should include fencing tokens or terms. Leadership changes must be observable and rare enough to avoid thrashing.

Example: *A scheduler elects one leader with term 57. Database updates reject jobs from term 56 after a network pause, even if the old scheduler still believes it is leader.*

C073 What are quorum reads and writes? [ ] [ ] [ ]

Answer: In a replicated system with `N` copies, a write quorum `W` and read quorum `R` can overlap when `R + W > N`, allowing a read to encounter a recent version under the system's version rules. Larger quorums improve consistency and durability but increase latency and reduce availability during failures. Quorum mathematics alone does not solve concurrent writes, clock issues, sloppy quorums, or conflict resolution.

Example: *With three replicas, writing to two and reading from two creates overlap. The reader compares versions and repairs a stale copy rather than accepting the first response blindly.*

C074 What is split brain? [ ] [ ] [ ]

Answer: Split brain occurs when separated nodes or regions both believe they are the active leader and accept conflicting operations. It can result from network partitions, stale leases, or unsafe failover. Prevention uses quorum-based leadership, fencing, and a rule that only one side can write protected state. Recovery must reconcile or discard conflicts according to business semantics and ensure the old leader is fenced before promotion.

Example: *Before promoting a standby database, the platform blocks the old primary's storage and network access. Otherwise both primaries could accept different writes under the same record IDs.*

C075 What are logical clocks? [ ] [ ] [ ]

Answer: Logical clocks represent ordering without trusting physical wall clocks. A Lamport clock increments on local events and advances when messages carry a larger value, establishing that causally earlier events have smaller timestamps. It cannot prove concurrency from equal or unrelated values. Logical clocks help order distributed logs and versions, while business timestamps can still be stored separately for display.

Example: *Service A sends event timestamp 12 to service B. B advances its logical clock beyond 12 before emitting a dependent event, preserving causal order despite clock skew.*

C076 What are vector clocks? [ ] [ ] [ ]

Answer: A vector clock tracks a counter for each participant or replica. Comparing vectors reveals whether one version causally follows another or whether the versions are concurrent and need conflict resolution. The metadata grows with participants and becomes difficult in highly dynamic systems, so practical implementations may limit or summarize it. Vector clocks identify conflict; they do not decide the correct business merge.

Example: *Two offline devices edit the same note from version `[3,5]`, producing `[4,5]` and `[3,6]`. Neither dominates the other, so the system preserves both or asks for a merge.*

C077 What is a lease? [ ] [ ] [ ]

Answer: A lease grants time-limited ownership. The holder must renew before expiry, and others may take ownership afterward. Leases recover from crashed holders without permanent locks, but clock assumptions, pauses, and network delay mean an old holder can continue briefly after expiry. Safe designs use conservative timing and fencing tokens at the protected resource. Longer leases reduce churn but slow failover.

Example: *A partition worker receives a 30-second lease and renews every 10 seconds. If renewal fails, it stops fetching work, while storage rejects stale-token commits after another worker takes over.*

C078 What is a fencing token? [ ] [ ] [ ]

Answer: A fencing token is a monotonically increasing number issued with ownership. Every operation on the protected resource includes the token, and the resource rejects a token older than the highest one already seen. This prevents a stale lock or lease holder from modifying state after a newer owner begins. Fencing must be checked where the write occurs; comparing tokens only in the worker leaves a race.

Example: *An object-generation job with token 102 replaces the media variant. A delayed worker with token 101 later tries to upload and is rejected rather than overwriting the current file.*

C079 Why is clock skew dangerous? [ ] [ ] [ ]

Answer: Distributed machines never have perfectly identical wall clocks. Skew can break token expiry, lease ownership, event ordering, billing windows, and last-write-wins conflict rules. Time synchronization reduces error but does not eliminate it. Durations should use monotonic clocks, ordering should use versions or logical clocks, and security checks should allow only a small bounded skew. Critical timestamps should record their authoritative source.

Example: *If one server is 90 seconds ahead, it may reject a newly issued token as expired. A bounded clock-skew allowance prevents that while monitoring alerts on the unhealthy host.*

C080 What is two-phase commit? [ ] [ ] [ ]

Answer: Two-phase commit coordinates an atomic decision across participants. In prepare, each participant promises it can commit and holds necessary resources. In commit, the coordinator tells everyone to finalize; on failure, participants may remain blocked while the coordinator recovers. It provides atomicity across supported systems but adds latency, locking, coordinator complexity, and poor availability during some failures. Many service workflows prefer sagas and idempotent compensation.

Example: *Two databases can prepare a transfer and both commit only after agreement. A long coordinator outage may leave locks held, which is unacceptable for many high-traffic microservice operations.*

9. Observability, Performance, and Capacity

C081 What are the four golden signals? [ ] [ ] [ ]

Answer: The four golden signals are latency, traffic, errors, and saturation. Latency measures request time, split by successful and failed requests. Traffic measures demand such as requests or events per second. Errors measure failed outcomes, including business failures when relevant. Saturation measures how close a constrained resource is to its limit. Together they show user impact and resource pressure, but still need endpoint, region, version, and dependency breakdowns.

Example: *A feed alert shows p99 latency rising, traffic unchanged, errors low, and database connections at 100 percent. That points toward waiting on a saturated pool rather than excess request volume.*

C082 What are the RED and USE methods? [ ] [ ] [ ]

Answer: RED focuses on request-driven services: Rate, Errors, and Duration. USE focuses on resources: Utilization, Saturation, and Errors. RED starts from user-facing work, while USE investigates CPU, memory, disks, network, pools, and queues. They complement rather than replace each other. A resource can have low average utilization but high saturation if work waits on a small pool or one hot shard.

Example: *RED shows upload duration worsening with stable rate and few HTTP errors. USE then reveals disk queue saturation on the media-processing volume.*

C083 Why use percentiles instead of average latency? [ ] [ ] [ ]

Answer: An average combines fast and slow requests and can hide the experience of the slowest users. Percentiles describe the value below which a percentage of observations falls: p50 is typical, p95 and p99 expose the tail. Percentiles should be measured over meaningful windows and segmented by endpoint, payload, region, and outcome. They do not identify the cause, so slow traces and saturation metrics remain necessary.

Example: *Ninety-nine requests at 100 ms and one at 10 seconds average about 199 ms, which looks acceptable. The p99 or maximum reveals the severe tail request that users notice.*

C084 What is distributed tracing? [ ] [ ] [ ]

Answer: Distributed tracing follows one request or event across service boundaries using a trace ID and parent-child spans. Each span records timing, outcome, and selected attributes, exposing where time and failures occur. Trace context must propagate through HTTP, messaging, retries, and asynchronous workers. Sampling controls cost, but important errors and high-latency traces should remain available. Traces explain individual paths; metrics still reveal overall frequency.

Example: *A 2-second feed trace shows 80 ms in the feed service, 40 ms in Redis, and 1.8 seconds waiting for recommendations, identifying the dependency that dominates latency.*

C085 How do metrics, logs, and traces differ? [ ] [ ] [ ]

Answer: Metrics are aggregated numeric time series suited to trends and alerts. Logs are timestamped records with detailed event context. Traces connect timed operations across a request path. Metrics answer whether and how often, traces answer where time went, and logs often explain what happened at a specific step. High-cardinality identifiers belong in logs or traces, not metric labels. All three need consistent service, version, region, and correlation fields.

Example: *A metric alerts on checkout errors, a trace finds the failing payment call, and a structured log records the provider error code and safe request identifier.*

C086 What does Little's Law explain? [ ] [ ] [ ]

Answer: Little's Law states that average items in a stable system equal arrival rate multiplied by average time in the system: `L = lambda x W`. It connects concurrency, throughput, and latency and helps estimate required in-flight capacity. It assumes a stable long-run system where arrival and completion rates match. If arrivals permanently exceed service rate, the queue grows without bound and the equation describes no steady state.

Example: *At 1,000 requests per second and 200 ms average response time, about 200 requests are in flight on average. A pool of 50 cannot support that concurrency without queueing.*

C087 What is Amdahl's Law? [ ] [ ] [ ]

Answer: Amdahl's Law shows that speeding up one part of a workflow gives limited total improvement when another part remains serial or unchanged. Optimization should target the largest contributor and consider the fraction that can run in parallel. Adding more workers provides diminishing returns when coordination, database locks, or one sequential step dominates.

Example: *If database work is 70 percent of request time, making JSON serialization ten times faster affects only the remaining fraction and barely changes end-to-end latency. Query optimization has more leverage.*

C088 What is capacity planning? [ ] [ ] [ ]

Answer: Capacity planning converts product demand into resource needs using peak requests, read-write mix, payload size, storage growth, concurrency, fan-out, background work, and failure headroom. It uses low, expected, and high scenarios rather than one precise forecast. Load tests find saturation points, while production data updates assumptions. The plan must include dependency quotas, scaling delay, deployments, and regional failover.

Example: *A launch estimate models 20,000 peak feed reads per second, 500 writes, 4 KB responses, a two-times burst, and enough spare capacity to lose one availability zone.*

C089 What is profiling? [ ] [ ] [ ]

Answer: Profiling measures where a program spends CPU time, allocates memory, blocks, performs I/O, or waits on locks. Sampling profilers have lower overhead and show hot call stacks, while instrumentation can provide exact timings with more cost. Profiles should come from representative load and be compared before and after a change. A CPU profile cannot explain time spent waiting on a remote database unless blocking and tracing data are included.

Example: *A Java flame graph shows excessive CPU in image metadata parsing, while allocation profiling reveals millions of temporary objects causing garbage-collection pauses.*

C090 What is tail latency? [ ] [ ] [ ]

Answer: Tail latency is the slow experience represented by high percentiles such as p95, p99, or p99.9. It often comes from cache misses, large payloads, garbage collection, lock waits, retries, slow shards, or one unhealthy zone. Fan-out amplifies tails because a request waiting for many parallel dependencies is affected by the slowest one. Tail reduction may require hedging safe reads, bounding fan-out, timeouts, and isolating slow cohorts.

Example: *A feed request fetching from twenty shards can be slow even when each shard is usually fast; one p99 shard response controls the whole request unless the design limits or bypasses it.*

10. Architecture and Integration Patterns

C091 How do monoliths, modular monoliths, and microservices differ? [ ] [ ] [ ]

Answer: A monolith deploys as one unit. A modular monolith keeps strong internal boundaries while retaining one deployment and often one transaction boundary. Microservices deploy independently and own their contracts and data, improving team autonomy and isolated scaling when boundaries are real. They add network failure, observability, compatibility, and distributed-data complexity. Service count is not the goal; independent change and failure ownership are.

Example: *Authentication and feed code can begin as modules in one Spring Boot application. Feed extraction is justified later if it needs independent scale, deployment, and data ownership.*

C092 When should communication be synchronous or asynchronous? [ ] [ ] [ ]

Answer: Synchronous calls provide an immediate response and simple request-response reasoning but couple caller latency and availability to the callee. Asynchronous messaging isolates failures, absorbs bursts, and supports multiple consumers, but introduces eventual consistency, retries, ordering, and workflow-state tracking. Keep only work required for the immediate user result synchronous and move independent side effects to durable asynchronous paths.

Example: *Creating a post synchronously validates and stores it. Search indexing, analytics, media variants, and notifications run asynchronously after a durable event.*

C093 What is event-driven architecture? [ ] [ ] [ ]

Answer: Event-driven architecture connects components through durable facts about completed state changes. Producers do not need to know every consumer, enabling independent reactions and replay. It also creates schema governance, duplicate delivery, ordering, lag, debugging, and eventual-consistency challenges. Events should have stable IDs, versions, timestamps, and ownership, and should not expose private database structures accidentally.

Example: *`UserFollowed` lets feed, recommendation, analytics, and notification systems react independently. A correlation ID links their outcomes back to the original follow request.*

C094 What is CQRS? [ ] [ ] [ ]

Answer: Command Query Responsibility Segregation separates the model used to change state from models optimized for reading it. Commands enforce invariants in the authoritative store, while events or change data build read models for specific queries. CQRS can improve scale and simplify complex reads, but adds duplication, eventual consistency, rebuild logic, and operational work. It is unnecessary for ordinary CRUD systems with manageable queries.

Example: *Follow commands update the relationship store, while an asynchronously built follower-summary table serves profile counts and relationship previews quickly for read traffic.*

C095 What is event sourcing? [ ] [ ] [ ]

Answer: Event sourcing stores the sequence of domain events as the authoritative record and derives current state by replay or snapshots. It provides history and supports new projections, but requires immutable schema evolution, event ordering, snapshotting, privacy handling, and careful business semantics. Correcting a mistake usually appends a compensating event rather than editing history. Event sourcing is different from merely publishing integration events after database updates.

Example: *An account balance can be derived from credit and debit events plus periodic snapshots. A mistaken debit is corrected with a reversal event that preserves the audit trail.*

C096 What are API composition and Backend for Frontend? [ ] [ ] [ ]

Answer: API composition combines data from several services into one response. A Backend for Frontend creates a client-specific API layer for mobile, web, or another interface. These patterns reduce client round trips and hide internal topology, but can create a latency fan-out and duplicated business logic. The layer should remain a bounded composer with parallel calls, deadlines, caching, and clear ownership.

Example: *A mobile BFF returns feed posts, viewer like state, and compact author summaries in one payload while leaving ranking and authorization inside their owning services.*

C097 What are sidecars and service meshes? [ ] [ ] [ ]

Answer: A sidecar is a companion process or container that provides networking, proxying, configuration, or observability beside an application. A service mesh uses such proxies plus a control plane to manage service-to-service identity, encryption, routing, retries, and telemetry. It standardizes cross-cutting behavior but adds resource cost, another failure layer, and configuration complexity. Application-level semantics such as idempotency and correct timeouts still matter.

Example: *A mesh provides mutual TLS and traffic splitting for feed-service canaries, while the feed application still decides whether a recommendation failure can use fallback.*

C098 What is the strangler pattern? [ ] [ ] [ ]

Answer: The strangler pattern replaces a legacy system incrementally. A routing layer directs selected capabilities or traffic to a new implementation while the rest remains on the old system. Data ownership, comparison, rollback, and retirement are handled one slice at a time. It lowers big-bang risk but temporary duplication can become permanent if each slice lacks completion criteria.

Example: *Profile reads move first to a new service, then updates, then media metadata. Routing can return one cohort to the legacy path if correctness metrics fail.*

C099 What is cell-based architecture? [ ] [ ] [ ]

Answer: Cell-based architecture divides a large service into mostly independent copies, each serving a subset of users or tenants with its own compute and data dependencies. A cell failure affects only its assigned population, reducing blast radius and enabling staged rollout. Routing, rebalancing, shared global services, and uneven cells add complexity. Cells should fail independently rather than depending on one hidden global bottleneck.

Example: *Users are assigned to one of fifty feed cells, each with service instances, cache, and database partitions. A bad release in one cell affects roughly two percent of users.*

C100 What are multi-tenancy isolation models? [ ] [ ] [ ]

Answer: Multi-tenancy can share tables with a tenant key, separate schemas, separate databases, or separate infrastructure. Greater sharing improves utilization and operational simplicity but increases noisy-neighbor and data-leakage risk. Stronger isolation increases cost and management overhead. Tenant identity must be enforced in authorization, queries, cache keys, queues, object paths, logs, and metrics regardless of storage model.

Example: *Small tenants share tables protected by tenant-scoped queries and row-level security, while a regulated enterprise receives a dedicated database and encryption key.*

11. Feed, Media, Search, and Product-Scale Patterns

C101 What are fan-out-on-write and fan-out-on-read? [ ] [ ] [ ]

Answer: Fan-out-on-write pushes a new post into follower feed stores when it is created, making reads fast but making celebrity writes enormous. Fan-out-on-read stores posts by author and merges followed authors during each read, making writes cheap but reads more expensive. A hybrid pushes normal accounts and pulls celebrity posts. Choice depends on follower distribution, freshness, ranking, and storage cost.

Example: *A user with 500 followers pushes 500 feed entries. An account with 50 million followers keeps the post in its author stream and is merged into feeds at read time.*

C102 What stages belong in a media-processing pipeline? [ ] [ ] [ ]

Answer: A media pipeline validates upload authorization and size, stores an immutable original, verifies checksum and file signature, scans or decodes safely, generates variants, extracts metadata, applies moderation, and publishes only ready outputs. Work is asynchronous, idempotent, observable, and retryable. Originals and processing state support reprocessing when algorithms change. Quarantine prevents partially processed or unsafe files from becoming public.

Example: *A photo upload returns `PROCESSING`; workers create thumbnails and WebP variants, moderation approves them, and the metadata changes to `READY` with versioned CDN URLs.*

C103 How is a search index kept in sync with a database? [ ] [ ] [ ]

Answer: The database remains authoritative while an outbox, change-data-capture stream, or durable event log sends changes to an idempotent indexer. Search is eventually consistent and needs lag monitoring, replay, deletion handling, and full rebuild capability. Direct best-effort dual writes can lose one side. Reindexing normally builds a versioned index, catches up changes, validates, and atomically switches an alias.

Example: *A committed post creates an outbox event. The indexer upserts document version 17 and ignores a later duplicate event carrying version 16.*

C104 What is an inverted index? [ ] [ ] [ ]

Answer: An inverted index maps each term to the documents containing it, often including positions, frequencies, and fields. It lets search find matching documents without scanning every document. Tokenization, language analysis, stemming, stop words, and field boosts determine what terms exist and how matches rank. Updates and merges add operational cost, and permission filters must be applied without leaking restricted documents.

Example: *The term `sunset` points to post IDs 4, 19, and 88 with caption positions. A query retrieves those candidates and ranks them using text relevance, freshness, and popularity.*

C105 What are candidate generation and ranking? [ ] [ ] [ ]

Answer: Candidate generation quickly retrieves a manageable set of potentially relevant items from follows, search indexes, embeddings, popularity, or rules. Ranking applies richer features and models to order that smaller set. Separating stages keeps latency and compute bounded; an expensive model cannot score every item in the corpus. Safety, privacy, and eligibility filters are correctness boundaries and should not depend solely on ranking scores.

Example: *A feed gathers 2,000 recent followed and recommended posts, filters blocked authors, then a model ranks the remaining candidates and returns the top 30.*

C106 How should a notification system be designed? [ ] [ ] [ ]

Answer: Notification producers emit durable intents, while a service applies preferences, deduplication, templates, channel policy, rate limits, scheduling, and provider delivery. Each logical notification has a stable ID, and provider retries are separated from permanent failures. Email, push, and in-app channels have different acknowledgement semantics. A global kill switch and per-user limits protect against storms.

Example: *`CommentAdded` creates one notification intent. The service suppresses it if the author muted comments, deduplicates retries, stores an in-app item, and sends one push through the provider.*

C107 How are large counters implemented? [ ] [ ] [ ]

Answer: Exact counters updated on one row can become a write hotspot. Systems may shard counters, batch increments, use atomic cache operations, or consume events into an eventually consistent aggregate. The relationship or event log remains the authoritative basis for reconciliation. Product requirements decide whether the displayed number can be approximate and how quickly it must reflect the initiating user's action.

Example: *Likes are stored uniquely by `(user_id, post_id)`. Counter workers aggregate events into shards and periodically reconcile the displayed total with authoritative relationships.*

C108 How is online presence represented? [ ] [ ] [ ]

Answer: Presence is an ephemeral signal derived from active connections, recent heartbeats, or device activity. It is usually eventually consistent because instantaneous global truth is expensive and disconnects can be ambiguous. Entries use TTLs so crashed clients disappear without an explicit logout. Privacy settings and scale often require coarse states such as active recently rather than exact timestamps.

Example: *A WebSocket gateway refreshes `user:42` presence every 20 seconds with a 60-second TTL. Friends may briefly see the user online after a network loss.*

C109 How are geospatial queries implemented? [ ] [ ] [ ]

Answer: Geospatial systems map latitude and longitude into searchable regions using structures such as geohashes, quadtrees, R-trees, or database spatial indexes. A coarse region finds candidates, then exact distance calculations filter and sort them. Cell size trades index breadth against false candidates. Boundaries, poles, wraparound, privacy, and location update frequency require care.

Example: *Nearby posts are retrieved from the user's geohash cell and adjacent cells, then filtered using great-circle distance to return items within five kilometers.*

C110 How does a URL shortener work? [ ] [ ] [ ]

Answer: A URL shortener creates a unique short key mapped to a validated destination. Reads are extremely frequent and cacheable, while writes need unique key generation, optional custom aliases, expiry, and abuse controls. Redirects can use 301 for stable permanent caching or 302 when destination changes and analytics matter. Key generation may use random strings, encoded numeric IDs, or allocated ranges.

Example: *Key `aB93x` maps to a long campaign URL in a durable store and cache. The redirect service reads locally, records analytics asynchronously, and prevents unsafe destinations.*

12. Security, Deployments, and Operational Safety

C111 What is the difference between authentication and authorization? [ ] [ ] [ ]

Answer: Authentication establishes who or what the caller is. Authorization decides whether that identity may perform a specific action on a specific resource. A valid login does not grant access to every object. Authorization should be enforced server-side near the protected operation and include tenant, ownership, role, relationship, and policy state. Cached responses must preserve the same access boundary.

Example: *A JWT authenticates user 42. The post service still checks that user 42 owns private post 99 or has explicit access before returning it.*

C112 How do sessions, JWTs, and OAuth differ? [ ] [ ] [ ]

Answer: A server session stores state centrally and gives the client an opaque session ID, making revocation straightforward but requiring shared session storage. A JWT is a signed self-contained token that services can verify locally, reducing lookups but making revocation and stale claims harder. OAuth is an authorization framework for delegated access; it can issue JWT or opaque tokens. Short lifetimes, secure storage, audience checks, and key rotation remain necessary.

Example: *A user signs in through OAuth, receives a short-lived access JWT, and uses a rotating refresh token stored and revocable on the server.*

C113 What is encryption in transit and at rest? [ ] [ ] [ ]

Answer: Encryption in transit protects data moving across networks, usually with TLS, and authenticates the endpoint through certificates. Encryption at rest protects stored disks, objects, databases, and backups if media or snapshots are exposed. Neither replaces authorization: an authenticated service can still request data it should not access. Key ownership, rotation, audit, and separation from encrypted data are central to the design.

Example: *Mobile traffic uses HTTPS, database volumes and object storage use managed encryption keys, and a separate key service controls access and records decryption operations.*

C114 What is secrets management? [ ] [ ] [ ]

Answer: Secrets management stores and distributes credentials, signing keys, certificates, and tokens without embedding them in code, images, logs, or ordinary configuration files. Access uses workload identity and least privilege, and secrets rotate automatically with audit trails. Applications should read secrets securely and handle rotation without restart where practical. A secret exposed in logs must be revoked, not merely deleted from one file.

Example: *A Spring service obtains a short-lived database credential from a secret manager using its runtime identity instead of loading a permanent password committed in `application.yml`.*

C115 What is defense in depth? [ ] [ ] [ ]

Answer: Defense in depth uses several independent controls so one mistake does not expose the system. Network restrictions, authentication, authorization, input validation, rate limits, isolation, encryption, audit, and monitoring each address different failure paths. Layers should not all rely on the same assumption or credential. More controls are useful only when ownership and failure behavior remain understandable.

Example: *A private admin API requires network access, service identity, an admin role, object-level policy, rate limits, and immutable audit logging. A routing mistake alone does not grant access.*

C116 What makes an audit log trustworthy? [ ] [ ] [ ]

Answer: An audit log records security- or business-sensitive actions with actor, action, target, time, result, policy context, and correlation ID. It should be append-oriented, protected by separate permissions, retained according to policy, and monitored for gaps. It avoids unnecessary secrets and personal content. Tamper evidence may use immutable storage or cryptographic chaining, but complete reliable delivery and controlled access are equally important.

Example: *Changing an account email writes an audit event to a separate protected store that ordinary application administrators can read but cannot edit or delete.*

C117 What is an expand-and-contract database migration? [ ] [ ] [ ]

Answer: Expand-and-contract changes schemas while old and new application versions coexist. Expand adds backward-compatible structures, then code writes and reads both forms as needed. Historical data is backfilled in bounded restartable batches. After validation and the rollback window, constraints change and obsolete structures are removed. Destructive changes are delayed until no deployed reader depends on the old format.

Example: *Add nullable `display_name`, deploy dual-write code, backfill existing users, switch reads, enforce non-null, then later remove the old `first_name` and `last_name` dependency.*

C118 How do rolling, blue-green, and canary deployments differ? [ ] [ ] [ ]

Answer: Rolling deployment replaces instances gradually within one environment. Blue-green maintains complete old and new environments and switches traffic, enabling fast rollback at higher cost. Canary sends a small representative share to the new version and expands only when guardrails pass. Database compatibility and external side effects can limit rollback for every strategy. Health, latency, errors, business outcomes, and downstream pressure should drive progression.

Example: *A feed release goes to 1 percent, 5 percent, 20 percent, and 50 percent before full rollout, automatically stopping if p99 latency or database connections regress.*

C119 What are feature flags? [ ] [ ] [ ]

Answer: Feature flags separate code deployment from feature exposure. They support gradual rollout, experiments, tenant targeting, and emergency disablement. Flags need owners, defaults, audit, consistent evaluation keys, and removal dates. They cannot make an incompatible protocol safe: consumers must understand both forms before producers send new data. Too many permanent flags create combinatorial testing and operational confusion.

Example: *New feed ranking ships dark, then enables for employees, one region, and ten percent of users. A kill switch restores chronological ranking without redeploying code.*

C120 What is a blameless postmortem? [ ] [ ] [ ]

Answer: A blameless postmortem explains what happened, user impact, timeline, contributing technical and organizational conditions, detection and response gaps, and what worked. It avoids punishing individuals for reasonable decisions made with available information, while still assigning clear action owners and dates. Actions should cover prevention, detection, containment, and recovery and later be verified. Blameless does not mean consequence-free; it means learning focuses on system improvement rather than fear.

Example: *After a retry storm, actions may add retry budgets, dependency dashboards, a circuit-breaker test, and an updated runbook, each with an owner and tracked completion date.*

CONCEPT ANSWER SHAPE

1. Define the concept plainly. **2.** Explain the problem it solves. **3.** State the main trade-off or failure mode. **4.** Give one concrete system example. **5.** Say when you would not choose it.

MORNING CONCEPT REVIEW LOG

Record the concepts you explained without notes and the missing idea or weak example to repair before the next recall.

Date	Concept IDs	Missing idea / better example	/5